FH University of Applied Sciences TECHNIKUM WIEN

Paperless

Mid-Term Submission
SWEN 3 Sem Project

Aslan Özugur Berceste Yagmur
Tokel Zeynep Bera

FH University of Applied Sciences TECHNIKUM WIEN

Repository References: https://github.com/bercesteyagmur/paperless-swen3.git

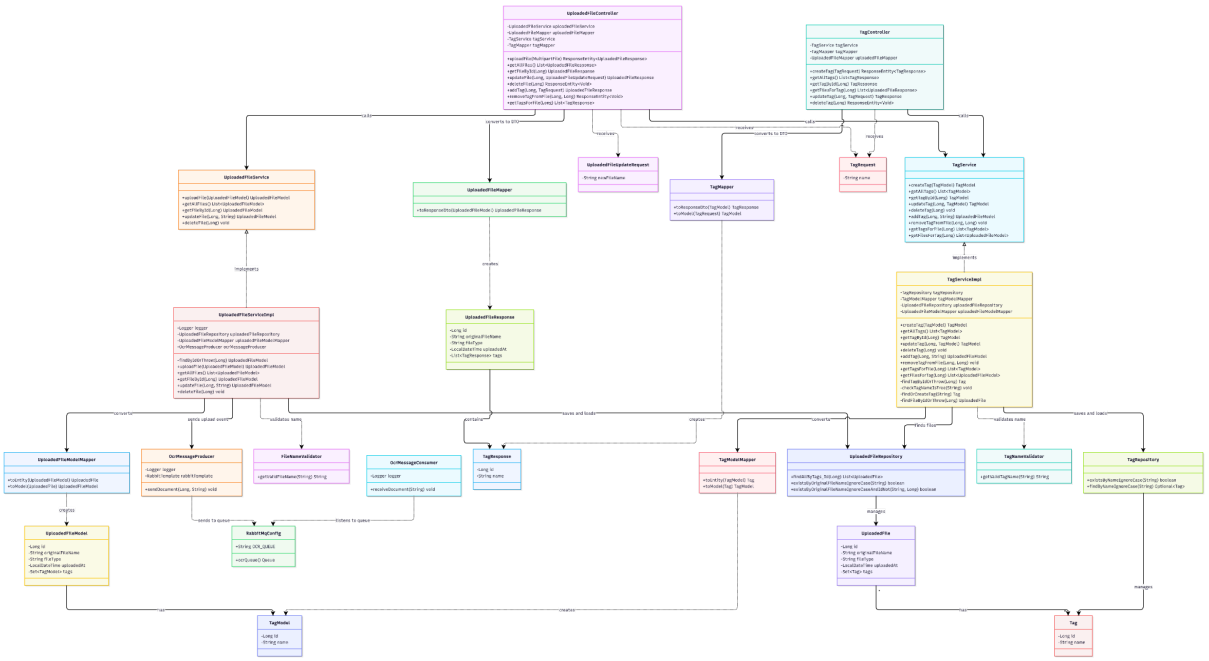
- README.md is for to show how to start the project with Docker Compose and how to run the tests
- CONTRIBUTING.md is for our rules for branches, pull requests and commits
- docker-compose.yml is for the setup PostgreSQL, RabbitMQ, backend and the frontend
- api-tests/: Postman collection and curl script to test all endpoints
- Swagger UI: http://localhost:8081/swagger-ui/index.html (only works when the application is running)

University of Applied Sciences
FH TECHNIKUM WIEN

1.Diagrams

1.1 Backend UML Diagram

This UML class diagram shows the main classes and how they are connected.



Our backend is built with Spring Boot and uses a layered architecture.
We have three layers: controller layer, service layer and data access layer. Each layer can only talks to the layer below it.

Controller layer has two classes, UploadedFileController and TagController. They receive the HTTP requests from the frontend. They call the services and use mappers to convert the data into DTOs.

Service layer has two interfaces, UploadedFileService and TagService. Their implementations are UploadedFileServiceImpl and TagServiceImpl.

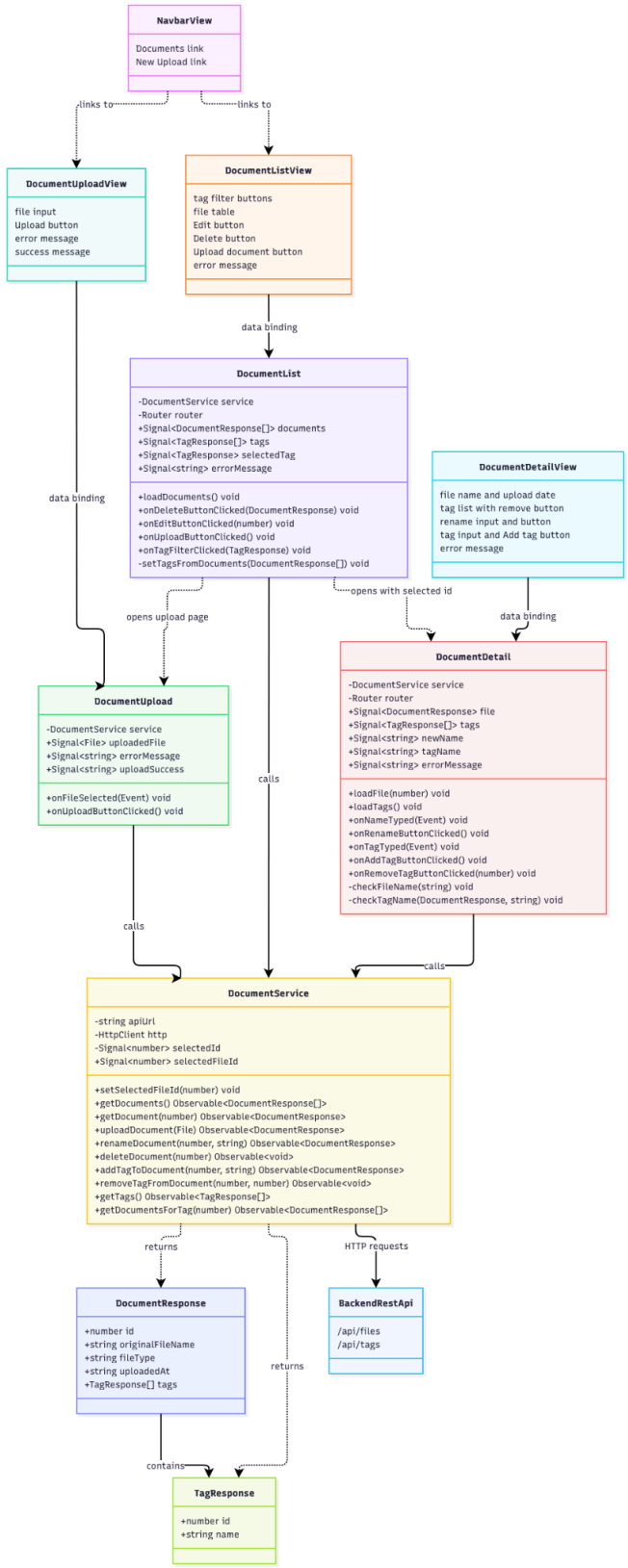
Data access layer has two entities, UploadedFile and Tag. They are saved in the PostgreSQL database.

For the database access we use UploadedFileRepository and TagRepository. Both extend JpaRepository, so we get the basic CRUD methods automatically.

We use three types of objects for the data. Entities are used for the database, models are used in the service layer and DTOs are used for the REST API in controller. Because of this,

the controller never sees an entity and the repository never sees a DTO. This way a change in the database does not directly change the API.

1.2 Frontend UML Diagram



Our frontend is built with Angular and follows the MVVM pattern. UML diagram shows how the View, the ViewModel and the Model are connected.

The file list is the start page, and from there the user can open the upload page with the "Upload document" button.

View is the HTML. We have three HTML: DocumentUploadView, DocumentListView and DocumentDetailView. There is also a NavbarView which links to the upload page and the file list. Views only show data and the buttons.

The ViewModel is the component class of each page: DocumentUpload, DocumentList and DocumentDetail. It works as a bridge between the View and the Model. The View never talks to the service directly.

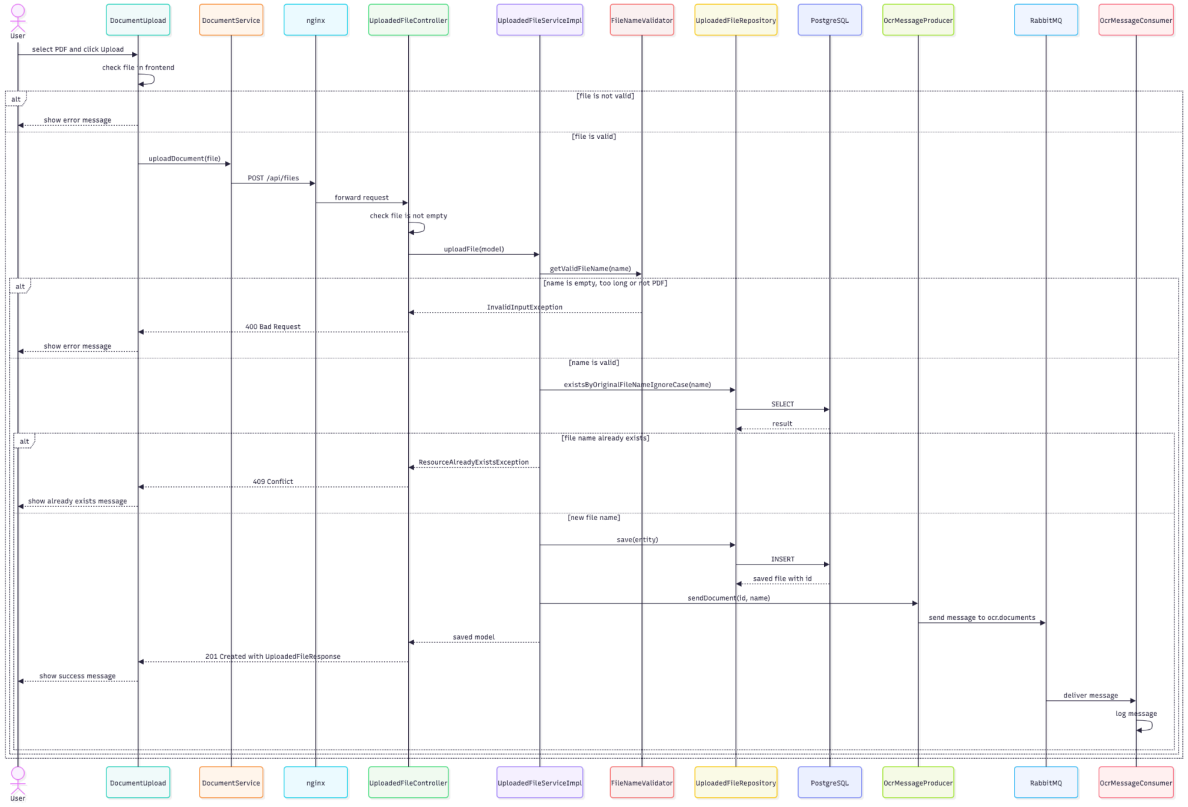
View only talks to its ViewModel, never with Model. When the page opens, ViewModel gets the data from the Model through DocumentService and saves it in Angular signals.

View shows this data with data binding. When the user does something, like clicking a button or typing a name, View calls a method in the ViewModel, for example onUploadButtonClicked() or onRenameButtonClicked(). ViewModel checks the input, sends the request to the Model and then updates the signals with the new data. Because of the data binding, the View changes automatically and immediately.

The Model part is DocumentService together with the interfaces DocumentResponse and TagResponse. DocumentService is the only class that talks to the backend. It sends HTTP requests to the REST API at /api/files and /api/tags and returns the data as Observables.

DocumentResponse and TagResponse are the frontend versions of our backend DTOs. In the backend, the controller sends UploadedFileResponse and TagResponse as JSON. In the frontend, we use these two interfaces to read this JSON. They have the same fields as the backend DTOs, for example id, originalFileName, fileType, uploadedAt and tags. Because of this, the data from the API fits directly into them and we can use it in the ViewModel without converting it.

1.3. Sequence diagram



This sequence diagram shows what happens step by step when a user uploads a PDF.

Frontend

User selects a file on the upload page and clicks "Upload". Before anything is sent to the backend, DocumentUpload component checks the file. The file must be selected, it cannot be empty, the name can be up to 100 characters and it must end with .pdf. If one of these checks fails, the user sees an error message and no request is sent. If the file is valid, the component calls uploadDocument() in DocumentService. The service puts the file into a FormData object and sends it as a multipart POST request to /api/files.

nginx

This request first goes to nginx. nginx serves the Angular frontend, and every request that starts with /api is forwarded to the backend container on port 8081.

FH University of Applied Sciences TECHNIKUM WIEN

Controller

UploadedFileController receives the file. It checks again that the file is not empty. Then it creates an UploadedFileModel with the original file name and the file type, and gives it to UploadedFileServiceImpl.

Service and validation

The service first calls FileNameValidator. The validator removes spaces at the beginning and end of the name and checks that the name is not empty, not longer than 100 characters and ends with .pdf. If not, it throws an InvalidInputException. Our GlobalExceptionHandler catches it and returns 400 Bad Request.

Next, service asks UploadedFileRepository if a file with the same name already exists. This check ignores upper and lower case. If the name exists, the service throws an exception and user gets 409 Conflict. Then frontend shows a message that the file already exists.

Saving the file

If everything is fine, the service sets the upload time and converts the model into an entity with UploadedFileModelMapper. The repository saves the entity in PostgreSQL, and the database creates a new id. Service writes a log message.

Sending to RabbitMQ

After saving, service calls OcrMessageProducer. The producer sends a message with the file id and the file name to the RabbitMQ queue "ocr.documents". If RabbitMQ is not available, the producer throws a MessagePublishException and the user gets 503 Service Unavailable. Because the service method is transactional, the saved file is rolled back in this case. This way we do not have files in the database that were never sent to OCR.

Response

Saved entity is converted back into a model and then into an UploadedFileResponse DTO. Controller returns it with 201 Create and frontend shows a message that the file was uploaded.

2. Design Decisions

2.1. Layered Architecture and MVVM

We split the backend into three layers and each class has one job. Controllers only handle HTTP, the services contain the business logic and the repositories talk to the database. In the frontend we use the MVVM pattern for the same reason. View only shows the data, the ViewModel handles the user actions and the Model talks to the backend.

2.2. Entities, Models and DTOs

We did not want to send our database entities directly to the frontend. That is why we have three kinds of objects: entities for the database, models for the business logic and DTOs for the API.

Because of this, we can change the database without changing what the frontend receives. To convert between these objects, we use MapStruct. We only write the mapper interfaces, and MapStruct creates the mapping code for us automatically.

2.3. Validation in Frontend and Backend

We check the user input first in the frontend. On the upload page, the user must choose a file, the file cannot be empty, it must be a PDF and its name can be up to 100 characters. On the detail page, the new file name cannot be empty or longer than 100 characters. A tag name cannot be empty or longer than 50 characters and the same tag cannot be added to a file twice. If something is wrong, the user sees an error message right away and no request is sent to the backend.

To be sure, backend checks again with FileNameValidator and TagNameValidator. This is important because API can also be called without the frontend for example with Postman. Backend also checks if a file or tag with the same name already exists in the database. In that case it returns 409 Conflict and the frontend shows a message that the name already exists.

3. Testing Strategy

We test the backend on different levels. Most of the tests are unit tests for the business logic because most of the rules are in the service layer.

All tests use an H2 in-memory database, so PostgreSQL does not need to run. We can start all tests with ./mvnw clean test.

3.1. Service Tests

For UploadedFileServiceImplTest and TagServiceImplTest we use Mockito to replace the repositories and mappers with mocks. This way we only test the business logic in the service. We test normal cases like uploading or renaming a file, and also error cases like a wrong file name or a duplicate name.

3.2. Repository Tests

For the repository tests we use @DataJpaTest with H2. We check that saving, finding, updating and deleting works. We also test our own queries, for example finding all files with a specific tag.

3.3. Controller and Messaging Tests

In UploadedFileControllerTest we check that an empty file is rejected. In OcrMessageProducerTest we check that the right message is sent to the OCR queue after an upload.

3.4. API Tests

To test the API, we use the Postman and the curl script in the api-tests folder. They upload a sample PDF and call all endpoints one by one.

3.5. Validation Tests

Most of our validation tests are in the service tests. For file names, we test that an empty name, a name that is not a PDF and a name that already exists are rejected. For tags, we test empty names, names longer than 50 characters and duplicate names. We also check that the right status code comes back, for example 400 Bad Request for a wrong input and 409 Conflict for a duplicate name. In UploadedFileControllerTest we check that an empty file cannot be uploaded.

FH University of Applied Sciences TECHNIKUM WIEN

4. Additional Use Case

As our own use case, we added tags to documents. A user can give one or more tags to a file, for example "invoice", "university" or "2026" and then filter the file list by these tags. This makes it easier to find documents when there are many files.

What the user can do

1. Add a tag to a file on the detail page. The user can type a new tag or choose an existing one from the suggestions.
2. Remove a tag from a file.
3. See the tags of every file in the file list.
4. Filter the file list by clicking on a tag. "All" shows all files again.

For this use case we added a new entity, Tag, with an id and a unique name.

Between UploadedFile and Tag there is a many to many relationship because one file can have many tags and one tag can belong to many files.

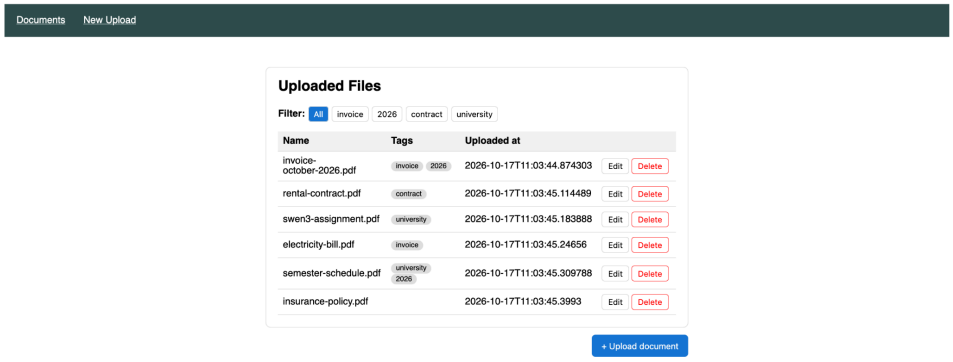


Figure 1: File list page with all uploaded files and their tags



Figure 2: File list filtered by the tag "invoice"

FH University of Applied Sciences TECHNIKUM WIEN

5. CI Pipeline

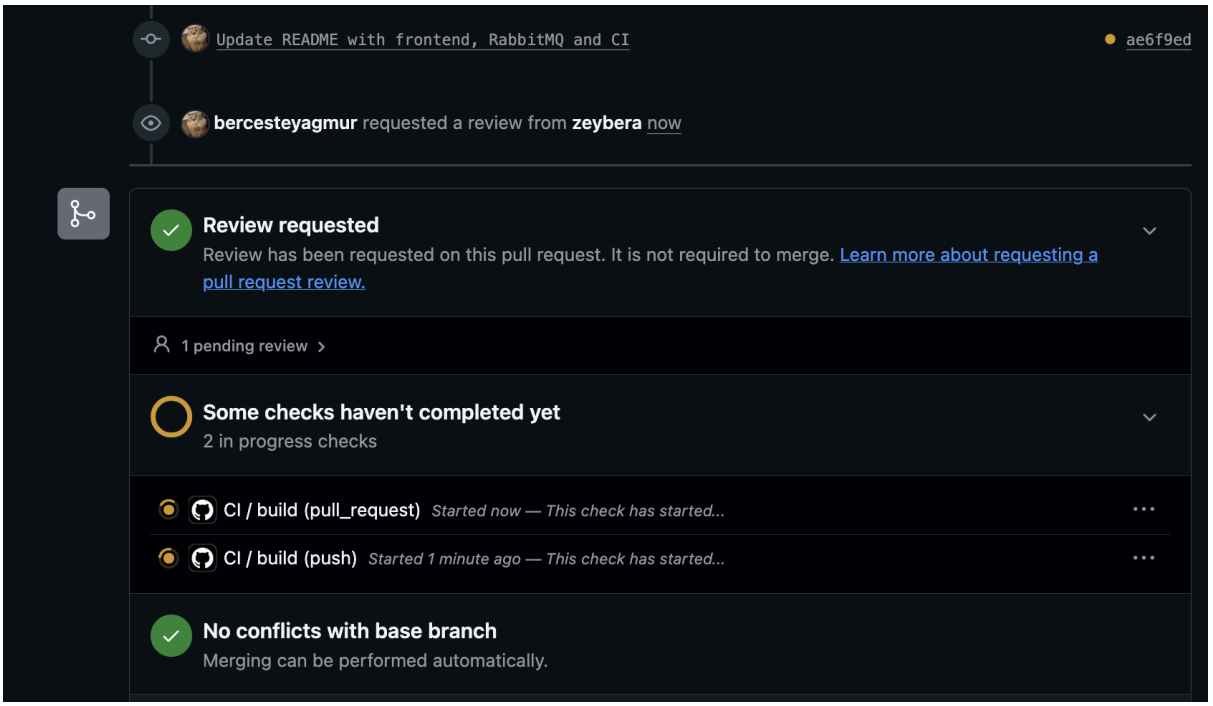
We use GitHub Actions for continuous integration.

Pipeline is defined in .github/workflows/ci.yml and runs automatically on every push and every pull request.

The pipeline has these steps:

1. It checks out the code from the repository.
2. It installs Java 25.
3. It runs all backend tests with ./mvnw test.
4. It builds all Docker images with docker compose build.

If a test fails or a Docker image cannot be built, the pipeline fails and we see a red on the pull request on GitHub. This way we notice problems before we merge into main.



6. Lessons Learned

1. Working with tickets helped us a lot. Every task had its own issue and its own branch, so we always knew who was working on what. At the start we did not have clear rules, so we wrote CONTRIBUTING.md. After that, every branch had the issue number in its name.

2. We had merge conflicts when a branch stayed open for too long. For example, the mapper branch and the tags branch were behind main, and we had to merge main into them first. We learned that it is better to keep branches small, merge them quickly and update them from main more often.

3. At first we wrote the mapper by hand and had no separate business layer. Later we added business models and replaced the mapper with MapStruct. We learned that a clear structure from the beginning saves a lot of refactoring later.

4. Pull requests with reviews were useful. A teammate saw problems that the author did not see.

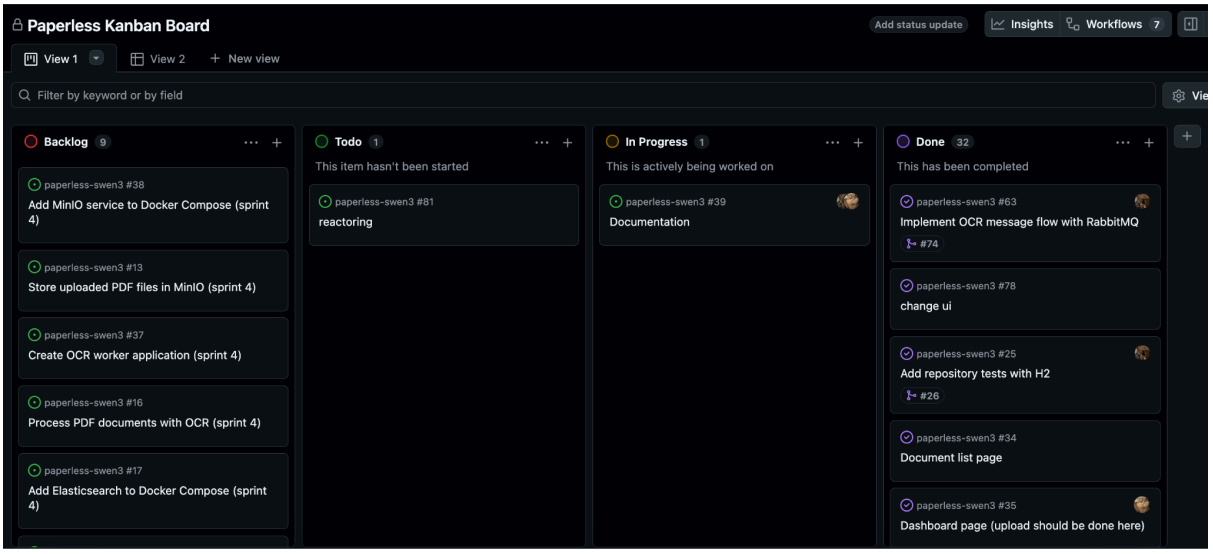
5. At the beginning, having entities, models and DTOs felt too much. But later we understood why they are important. When we added tags, we could change the database and the business logic without changing what the frontend receives. Each layer only works with its own objects, so a change in one layer does not break the others.

6. We learned that validation in the frontend is not enough. The backend has to check everything again.

7. Docker Compose made our work easier. Everyone could start the whole project with one command.

FH University of Applied Sciences TECHNIKUM WIEN

7. Kanban board: https://github.com/users/bercesteyagmur/projects/1



We planned and tracked our work on a Kanban board in GitHub Projects called "Paperless Kanban Board". The board has four columns: "Backlog", "Todo", "In Progress" and "Done".

In the Backlog we keep the tasks for the next sprints. We write the sprint in the issue title, for example "Create OCR worker application (sprint 4)". When we plan the next sprint, we move these issues to "Todo".

When someone starts a task, they move the issue to "In Progress" and create a new branch from main. The branch name has the issue number in it, like feat/63-ocr-message-flow. When the work is done, they open a pull request that is linked to the issue. A teammate reviews it, and after the merge the issue is moved to "Done".